

SE AI Agent Index: A Dataset of Software Engineering Practices for Agentic AI Systems

Mustahid Hasan
William & Mary, USA
mhasan02@wm.edu

Antu Saha
William & Mary, USA
asaha02@wm.edu

Oscar Chaparro
William & Mary, USA
oscarch@wm.edu

Abstract—Agentic AI systems have rapidly expanded across domains through autonomous reasoning, tool use, and task execution capabilities. As organizations increasingly deploy these systems in production, the software engineering (SE) practices behind their development and maintenance become valuable knowledge for industry practitioners, researchers, and educators. However, this knowledge remains scattered across engineering blogs, technical documentation, repositories, and practitioner reports, making it difficult to understand how teams engineer agentic systems in practice. To address this gap, we present the *SE AI Agent Index*, a structured dataset documenting software engineering practices for 13 industrial AI agents across seven dimensions, including software methodologies, testing, code review, CI/CD, issue resolution, communication, and documentation. The dataset contains 608 information items extracted from publicly available sources and organized into practices, tools, stakeholders, and other relevant information. We constructed the dataset using a semi-automated methodology that combines LLM-assisted information extraction, manual synthesis and refinement, and independent validation to ensure that all indexed information is traceable to verifiable evidence. To demonstrate its utility, we present usage scenarios showing how the dataset can support industry practitioners, researchers, and educators in understanding and studying the engineering of modern AI agents.

I. INTRODUCTION

Artificial Intelligence (AI) agents—autonomous programs that plan multi-step workflows, invoke tools, and act on behalf of users with minimal human involvement—have rapidly moved from research prototypes to production deployments [1], [2], [3]. These agents have expanded across domains from scientific research and healthcare to finance, enterprise automation, and notably software engineering (SE), fundamentally altering how complex work is structured by automating development workflows such as code generation and program repair [1], [4].

Given the autonomy of agentic systems and the non-deterministic, data-driven nature of Large Language Models (LLMs), the process of building and maintaining these systems likely differs from that of traditional software systems [5], [6]. Unlike traditional applications, agentic systems must operate under uncertain model behaviors, coordinate tool interactions, manage long-running workflows, and remain reliable in dynamic environments. These characteristics introduce new software engineering challenges related to testing, observability, deployment, debugging, safety, and maintenance [5]. As these systems continue to grow in capability and adoption, engineering teams must continuously adapt their development and operational practices to address these challenges.

However, despite the rapid industrial deployment of AI agents, the software engineering practices adopted by organizations building and maintaining these systems remain largely unknown to the research community.

Existing efforts to document AI systems and agents, including model cards [7] and system cards [8], primarily focus on documenting model characteristics, intended uses, risks, and governance mechanisms. Similarly, the AI Agent Index [9] documents agents by systematically collecting evidence about their technical capabilities (*e.g.*, tool use, coding assistance, web interaction, and autonomy), safety measures, deployment characteristics, and governance structures. These resources provide valuable insights into what AI systems and agents do, how they are evaluated, and how organizations manage their risks and deployment. However, they do not document the software engineering practices of the teams who build and maintain these systems, including the testing methodologies, CI/CD pipelines, code review processes, issue-resolution workflows, and documentation practices followed throughout development and operation.

To fill this gap, we introduce the *SE AI Agent Index*: the first structured dataset that documents the software engineering practices used to develop, maintain, and evolve deployed AI agents. The dataset contains 608 information items collected for 13 industrial AI agents spanning chat-based agents with tool access, browser-based agents, and enterprise workflow agent platforms. These information items capture practices, tools, stakeholders, and other relevant information across seven SE dimensions: methodologies, code review, testing, CI/CD, issue resolution, communication, and documentation. To construct the dataset, we adopted a semi-automated methodology that combines LLM-assisted information extraction from publicly available sources and manual qualitative analysis and validation. The resulting dataset provides evidence-backed and traceable documentation of how AI-agent development teams engineer, test, deploy, and maintain AI agents in practice.

Beyond serving as a curated repository of SE practices, the *SE AI Agent Index* provides a foundation for research, education, and practice in AI-agent engineering. It enables comparative studies of deployed agents, supports evidence-based teaching through real-world case studies, and offers practitioners insights into current engineering practices. Together, these applications help advance the understanding of how AI agents are developed, deployed, and maintained in practice.

II. THE SE AI AGENT INDEX

The *SE AI Agent Index* is a collection of *practices, tools, stakeholders*, and *other* information about the development, maintenance, and evolution of 13 industrial AI agents. The data is structured and indexed in multiple *SE dimensions* and is stored as a spreadsheet table, with intermediate files and supporting tooling to retrieve and access the *information items* stored in the index. These items were extracted from publicly available sources, including agents’ developer blogs, project repositories, and official documentation.

This section discusses the structure of the index, while Section III details the methodology used to construct and validate it. Section IV illustrates how the index can be used in practice by industry practitioners, researchers, and educators, through concrete examples of indexed information items.

A. AI Agents

The 13 AI agents of our dataset are a subset of the 30 agents documented in the 2025 AI Agent Index [9]. This index contains information about the origins, design, capabilities, ecosystem, and safety features of 30 state-of-the-art AI agents based on publicly available information and developer feedback. The 30 agents span three categories: chat-based agents with tool access, browser-based agents, and enterprise workflow agent platforms.

To ensure diversity of agents and domains, we randomly selected 4 to 5 agents from each category (13 agents in total): 4 chat-based agents with tool access (Anthropic Claude Code, Amazon Q Developer, Google Gemini CLI, Google Gemini); 5 browser-based agents (Browser Use, Opera Neon, Kimi Ok Computer, Perplexity Comet, Alibaba Mobile Agent); and 4 enterprise workflow platforms (Google Gemini Enterprise, Glean Agents, IBM Watsonx Orchestrate, HubSpot Breeze Studio). These agents cover various domains such as software development, browser automation, and enterprise business processes.

B. Information Items and Dimensions

Our dataset comprises specific facts (*i.e.*, information items) about an agent’s development and maintenance, across four categories: *practices, tools, stakeholders*, and *other* relevant information. These facts, extracted from publicly available sources and synthesized via qualitative analysis (described in Section III), are organized into seven dimensions:

- **SE Methodologies:** facts about overall software development/maintenance workflows and methodologies, including sprint organization, team size, and planning practices.
- **Code Review:** facts about how code review is performed, including pull request policies, approval requirements, and code/pull request quality checks.
- **Testing:** facts about how the agents are verified and validated, including test frameworks, coverage targets, and testing/evaluation strategies.
- **CI/CD:** facts about continuous integration/deployment (CI/CD) pipelines, including deployment processes, release practices, and rollback procedures.

- **Issue Resolution:** facts about the issue resolution process, including issue triage workflows, debugging approaches, and recurring engineering problems.
- **Communication and onboarding:** facts about agent developers’ internal communication channels and contributor onboarding processes.
- **Documentation:** facts about project documentation culture, knowledge-sharing blogs, and other related topics.

The index also contains contextual information about each agent, such as a general description of the agent and its features, whether it is open- or closed-source, programming languages, and meta-data such as # of contributors, releases, and stars.

Each information item in the index (*e.g.*, a specific testing practice or tool used by an agent’s development team) is traceable to supporting evidence. For each agent, extracted quotations (verbatim phrases from the sources) and source metadata are stored in a structured JSON file alongside the final spreadsheet index. Each item is uniquely identified and linked to its quotations and sources, enabling users to retrieve the original supporting evidence. This design facilitates auditing, verification, and future updates of the dataset while preserving transparency throughout the indexing process (see Section III).

C. Index Implementation and Tooling

The information items extracted and validated from the sources are stored in spreadsheet’s table (see `index.xlsx` or `index.pdf` in our dataset repository [10]), where each row corresponds to an agent, each column corresponds to an information category for a given dimension (*e.g.*, CI/CD practices or code review tools), and each cell includes a list of textual statements corresponding to the information items (each uniquely identified). Notably, each of the seven dimensions enclose four columns: *practices, tools, stakeholders*, and *others*.

The information items were synthesized and validated using the methodology described in Section III. The items are traceable to quotations extracted from the sources. These quotations and corresponding sources (their URL) are stored in a JSON file for each agent named `[agent_name]_se_index.json` and are also organized into the seven dimensions and four information categories (*practices, tools, etc.*). Each source is uniquely identified with a number and has two associated dates: the *creation date* (extracted from the source when available), and the *retrieved date* (indicates when we processed the source to extract the quotations).

To easily retrieve the quotations and information sources for a given information item (identified by “q-id”, *e.g.*, “q-2”), we developed a script that can be executed as: `./se_index.sh query <agent_name> <q-id>`.

D. Dataset Statistics

As of May 18, 2026, the dataset spans 13 agents and 608 information items across seven dimensions and information types (37 to 66 per agent, 47 on average). The index contains 260 information sources across all agents (8 to 43 per agent, 20 on average). The sources span official project engineering blogs and documentation (44%), GitHub repositories and issue track-

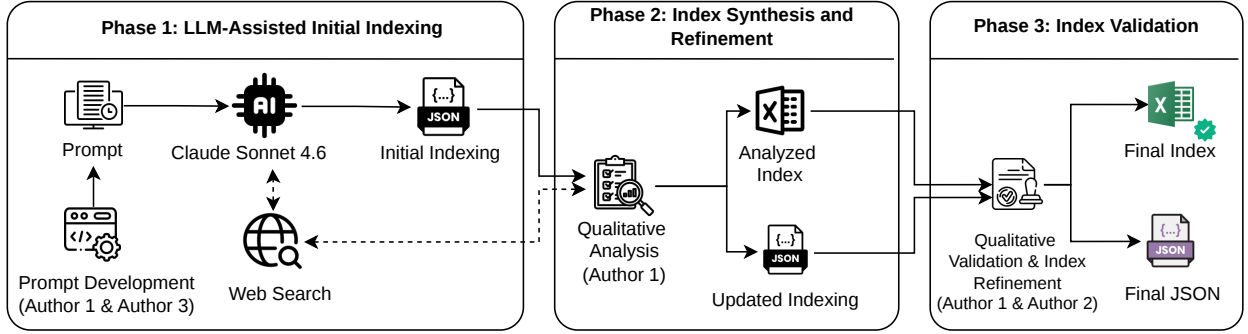


Fig. 1: Methodology for extracting, synthesizing, and validating agent engineering information items in the *SE AI Agent Index*.

ers (24%), and other sources (12%) such as technical papers, security reports, podcasts, and practitioner interviews. 93.5% of the 260 references are from the agents’ official documentation.

III. DATASET CONSTRUCTION METHODOLOGY

Our goal was to identify and systematically document information about the engineering and maintenance of the 13 AI agents in the index. To this end, we adopted a semi-automated approach that combines LLM-assisted information extraction with qualitative analysis and manual validation.

The overall process, shown in Figure 1, consisted of three phases: (1) generation of an initial index from publicly available sources, (2) synthesis and refinement of the extracted information, and (3) validation of the resulting dataset.

A. Phase 1: LLM-Assisted Initial Indexing

For each AI agent, we developed a structured prompt and executed it using CLAUDESONNET-4.6 through Claude Code CLI to generate an initial index in JSON format. The JSON schema mirrored the final dataset structure, organizing information into the seven SE dimensions and the four information categories described in Section II-B.

The prompt (see `index_prompt.md` in [10]) specified the indexing objective, agent name, output schema, definitions of the SE dimensions, and instructions for identifying relevant evidence from publicly available sources. We instructed the model to search (via its internal web search engine) and collect information from diverse source types, including project repositories, official documentation, engineering blogs, issue trackers, technical papers, practitioner interviews, and other publicly available materials. For each source, the model was asked to extract SE-related information, generate concise summaries, and provide supporting verbatim quotations linked to the original source.

The prompt was developed iteratively by the first and third authors. We initially defined the dimensions, then progressively refined the prompt by introducing detailed sub-dimensions, clarifying the distinction between product features and development practices, and adding constraints to reduce irrelevant information extraction. Throughout this process, we reviewed the generated outputs and refined the prompt accordingly with three of the 13 agents: Claude Code, Browser Use, and

Glean Agents. We also compared the quality of the generated indexes produced by GEMINI-2.5-PRO, GEMINI-3.1-PRO, and CLAUDESONNET-4.6. Based on consistency, completeness, and traceability of the extracted information, we selected CLAUDESONNET-4.6 for dataset construction.

B. Phase 2: Index Synthesis and Refinement

The initial JSON index for each agent was manually reviewed by the first author, who examined every generated summary, supporting quotation, and source reference to verify correctness and relevance. During this process, the author synthesized validated information items (*i.e.*, facts) into the final spreadsheet-based index. Each validated summary was converted into one or more information items and assigned to the appropriate dimension and information category (practices, tools, *etc.*). Information unrelated to SE practices, such as product features or end-user functionality, was excluded.

The author also verified that quotations were verbatim excerpts from the cited sources, checked whether quotations adequately supported the generated summaries, removed inaccessible or invalid sources, corrected extraction errors, and identified omitted information when present in the cited source. For dimensions or categories where the model reported that no relevant information was found, the author performed additional manual web searches to confirm the absence of publicly available evidence or to add and process new sources.

C. Phase 3: Index Validation

To improve accuracy and completeness, the second author independently reviewed the resulting index. Each information item was inspected against its supporting quotations and original sources.

The reviewer verified the correctness of the synthesized information, assessed whether quotations and sources provided sufficient evidence, and identified cases where additional details were required. The reviewer also focused on detecting irrelevant content, missing references, broken traceability links, and mismatches between information items and their supporting evidence. When necessary, the reviewer revisited the original sources to recover additional context or correct inaccuracies. Agent-level metadata included in the dataset was also validated during this stage by the reviewer. The

reviewer found 85.6% of the information items were correct. Both the reviewer and first author discussed and resolved the discrepancies in the remaining 14.5% of the items, which were mainly due to writing mistakes.

IV. INDEX USAGE SCENARIOS AND EXAMPLES

The *SE AI Agent Index* can be used by *developers* who build, deploy, and maintain AI agents; *researchers* who need to understand AI agent engineering in practice; and *educators* who need to teach how industrial AI agents are engineered and operated. This section presents representative examples from the index; source citations are omitted due to space constraints. The README file in the dataset repository [10] provides the same examples with their corresponding references.

A. AI Agent Developers

AI agent developers can use our index to:

1) **Assess internal engineering practices against high-maturity scenarios.** For example, development teams assessing or designing an overall testing strategy for their agents can learn from HubSpot’s Breeze Studio, whose developers adopted the *Testing Trophy* strategy, where they prioritized integration tests for their agent(s) as the best balance between confidence and maintenance cost while still using static analysis, unit tests, and end-to-end tests where appropriate.

2) **Identify tools for addressing AI-specific engineering challenges.** Compared to traditional software, AI agents introduce unique engineering challenges such as non-deterministic responses, hallucinations, and token cost growth. Our index can help developers identify which tools are used in mature projects to address these challenges. For instance, developers of IBM’s WatsonX Orchestrate use OpenTelemetry to trace and analyze agent trajectories and thus monitor and correct hallucinations and token usage.

3) **Improve infrastructure for building and executing AI agents.** Practitioners can learn from documented infrastructure practices in our index to adopt proven agent designs. Browser Use, for instance, describes two sandboxing strategies for their code-executing agents: (1) only risky operations such as code execution, shell commands, and file system access are isolated in a sandbox while the agent runs on the backend, or (2) the entire agent runs in a sandbox with no credentials and a proxy service holding all API keys and database credentials in every external request. They migrated from strategy 1 to 2 for better scalability and security. By indexing such practices, we enable developers at other organizations to evaluate and adopt the same architectural improvements.

4) **Strengthen security and reliability practices.** AI agents are susceptible to adversarial attacks (e.g., prompt injection or instruction override), which introduce security and compliance risks. Our index documents mitigation strategies for these risks, which other teams can learn and adopt. For example, IBM’s Watsonx Orchestrate Agent Development Kit (ADK) ships with a built-in red-teaming command that simulates 15+ attack types, including prompt injection and instruction override.

5) **Learn from operational impact.** Evidence drawn from

companies’ development blogs helps teams decide which AI-engineering practices are worth adopting, build a business case for investment, and prioritize changes that improve development speed. For instance, HubSpot rewrote its old code-review system (Sidekick) to *Aviator*; a Java-based agent framework for code review. The migration delivered concrete operational gains: reduced time-to-feedback by 90%.

B. Researchers

Researchers can use our index to:

1) **Perform cross-case studies of how organizations engineer and maintain agents.** This includes cross-case studies on various types of agents (e.g., browser-based, chat-based, and enterprise-level) across the seven dimensions detailed in Section II-B. This comparative analysis can help them evaluate AI-agent engineering practices to distinguish whether they are isolated to a specific company, unique to a certain agent category, or indicative of broader trends. For example, one trend is that 5 of 13 agents have adopted the *LLM-as-judge* approaches, supplementing traditional test suites: Browser Use uses Gemini 2.5 Flash as a judge for code reviews; HubSpot uses LLM-as-judge to evaluate the code review comments.

2) **Quantify the impact of engineering practices on agent development and operations.** Many SE practice decisions are made on intuition rather than empirical evidence. Researchers can analyze our index to identify such evidence or gather additional evidence from other sources. For instance, HubSpot’s CI/CD restructuring cut build times from 10–15 to 2–3 minutes using Kubernetes and build caching; Browser Use’s fast-iteration methodology produced 4,000+ commits in one year, with mistakes shipped, analyzed, and fixed forward as a deliberate engineering practice. Such documented outcomes give researchers a starting point for empirically quantifying which SE decisions most improve agent quality.

C. Educators

Educators can use our index to:

1) **Present case studies showing how industrial AI agents are built, tested, deployed, reviewed, documented, and operated at scale.** Without our index, educators can teach students that real-world agentic systems require disciplined engineering beyond prompt engineering. For instance, Glean illustrates testing and code review: agents are evaluated against curated datasets of real-world examples graded on binary sub-metrics (completeness, tone, groundedness), while dedicated pull request review agents scan code changes, flag risk areas, and produce prioritized checklists to keep review quality consistent at scale. Anthropic Claude Code illustrates rapid development and deployment: the team builds functional prototypes daily using parallel agents and iterative feedback and refinement rather than relying on heavy and static mocks/Figma designs and Product Requirement Documents (PRDs). This allows the team to ship up to 100 internal releases per day. These cases help students connect abstract SE concepts to the engineering practices of deployed AI systems.

2) **Support course modules or assignments.** The index can support comparative analysis, evidence-based argumentation, and student projects that ask learners to reason from real engineering examples. Our index dimensions (see Section II-B) naturally map to course modules on testing methodologies, CI/CD, code review, bug solving, software engineering methodology, developer communication, and more. Students can compare agents within each category, identify evidence gaps, and propose improvements backed by agent-specific examples.

V. LIMITATIONS AND THREATS TO VALIDITY

Construct validity. Public documentation varies in completeness across agents. Most information items in our index are supported by agent-specific evidence or a combination of agent- and organization-level sources; however, some practices are documented only at the organizational level. When agent-specific evidence was unavailable, we attributed organization-wide practices to an agent only when no contradictory evidence existed. This improves coverage but may misrepresent cases where teams adopt deviations from organizational processes.

Internal validity. The dataset was constructed using a semi-automated methodology that combines LLM-assisted extraction with manual qualitative analysis and validation. Although the structured schema, evidence traceability, and independent reviews reduce extraction and synthesis errors, some risk remains due to human interpretation and occasional LLM extraction inaccuracies. Furthermore, some practices may remain undocumented despite supplementary manual searches.

External validity. The dataset covers 13 agents selected from the 30 systems documented in the AI Agent Index. These agents are prominent industrial systems developed by organizations with mature engineering processes and extensive public documentation. The observed practices may not generalize to smaller organizations, less mature teams, or environments with limited public disclosure. The dataset should be interpreted as characterizing a subset of well-documented industrial AI agents rather than the entire AI-agent ecosystem.

Temporal validity. The dataset reflects evidence available up to May 18, 2026. Because AI-agent development practices evolve rapidly, some documented practices may change over time, and newer practices may not be captured. To support future updates and reproducibility, we make our infrastructure (prompt, tools, *etc.*) and preserve source references, quotations, and retrieval metadata (*e.g.*, creation and retrieval dates), enabling re-execution of the indexing and validation process.

VI. RELATED WORK

Prior work has examined agentic systems from several complementary perspectives. The AI Agent Index [9] documents the capabilities, deployment characteristics, and safety mechanisms of deployed AI agents. Wang *et al.* [11] analyzed ten widely used agent-development frameworks, identifying common engineering challenges and framework trade-offs. Other studies have focused on specific software engineering concerns. For example, Hasan *et al.* [12] identified ten testing patterns in open-source AI agents, including approaches for handling non-deterministic

behavior, while Alenezi [13] discussed how software engineering processes may need to evolve for agentic systems. More broadly, AI governance research has emphasized the importance of systematic documentation and transparency [14].

Despite these contributions, no prior work systematically documents how deployed AI agents are engineered and maintained across multiple software engineering dimensions. We address this gap by introducing the first dataset of software engineering practices used in the development and maintenance of deployed AI agents, enabling cross-agent analyses of agent engineering in practice.

VII. CONCLUSION

We introduced the *SE AI Agent Index*, the first structured dataset documenting the software engineering practices used to develop and maintain deployed AI agents. The dataset covers 13 industrial AI agents and was constructed through a semi-automated methodology that combines LLM-assisted extraction with manual qualitative analysis and validation. By preserving traceability to supporting evidence, the index provides a transparent and reproducible foundation for studying how AI agents are engineered in practice. We envision the dataset supporting future empirical research, educational activities, and industrial benchmarking as AI-agent engineering continues to evolve.

REFERENCES

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Lin, T. Gui, Y. Qu, X. Qiu, X. Huang, and Z. Wei, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [4] A. Soni, A. Kumar, R. Arora, and R. Garine, "Integrating ai into the software development life cycle: best practices, tools, and impact analysis," *Tools, and Impact Analysis (June 10, 2023)*, 2023.
- [5] A. E. Hassan, H. Li, D. Lin, B. Adams, T.-H. Chen, Y. Kashiwa, and D. Qiu, "Agentic software engineering: Foundational pillars and a research roadmap," *arXiv preprint arXiv:2509.06216*, 2025.
- [6] F. Fournier, L. Limonad, and Y. David, "Agentic ai process observability: Discovering behavioral variability," 2025. [Online]. Available: <https://arxiv.org/abs/2505.20127>
- [7] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *FAccT'19*, 2019, pp. 220–229.
- [8] F. Guroy and I. A. Kakadiaris, "System cards for AI-based decision-making for public policy," *arXiv preprint arXiv:2203.04754*, 2022.
- [9] L. Stauffer, K. Feng, K. Wei, L. Bailey, Y. Duan, M. Yang, A. P. Ozisik, S. Casper, and N. Kolt, "The 2025 AI agent index: Documenting technical and safety features of deployed agentic AI systems," *arXiv preprint*, 2025, available: <https://aiagentindex.mit.edu>.
- [10] "Dataset repository," <https://github.com/musta55/se-ai-agent-index>, 2026.
- [11] Y. Wang, X. Xu, J. Chen, T. Bi, W. Gu, and Z. Zheng, "An empirical study of agent developer practices in AI agent frameworks," *arXiv*, 2025.
- [12] M. M. Hasan, H. Li, E. Fallahzadeh *et al.*, "An empirical study of testing practices in open source AI agent frameworks and agentic applications," *Empirical Software Engineering*, 2026.
- [13] M. Alenezi, "Rethinking software engineering for agentic AI systems," *arXiv preprint arXiv:2604.10599*, 2026.
- [14] A. Winecoff and M. Bogen, "Improving governance outcomes through ai documentation: Bridging theory and practice," in *CHI*, 2025, pp. 1–18.